Java Backend Architecture

Interview Series

Chapter 7.2
Maven Dependency Management

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 7.2 — Maven Dependency Management

Introduction

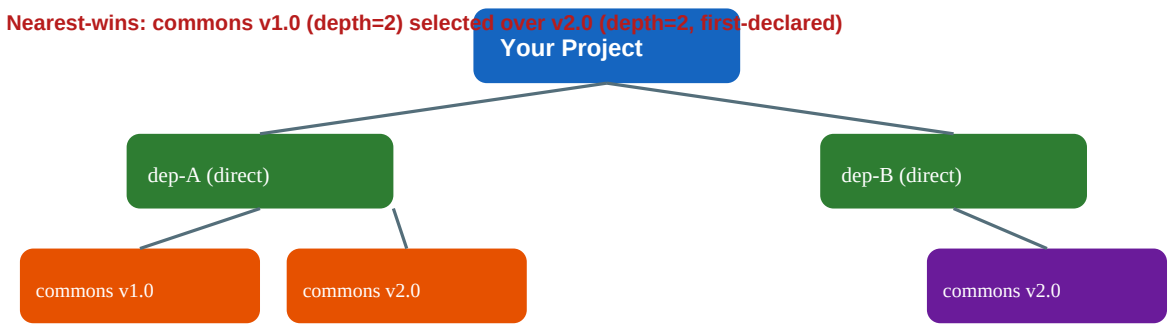
Maven's dependency management system is one of its most powerful features. It handles transitive dependencies automatically, provides a rich scope system to control classpath visibility, supports Bill of Materials (BOM) imports for centralised version management, and offers tools to analyse and resolve dependency conflicts in large enterprise projects.

Dependency Scopes

Scope	Main CP	Test CP	Runtime	Package	Description
compile	CP	CP	RT	Pkg	Default scope — classpath everywhere + packaged
provided	CP	CP	—	—	Provided by container (e.g., servlet-api)
runtime	—	CP	CP	Pkg	Not needed for compile, but needed at runtime
test	—	TC	TR	—	Only for test compile & run
system	CP	CP	RT	—	Explicit local JAR — not portable
import	—	—	—	—	BOM import — only in dependencyManagement

```
<!-- pom.xml dependency examples for each scope --> <dependencies> <!-- compile (default) -->
<dependency> <groupId>org.springframework</groupId> <artifactId>spring-core</artifactId>
<version>6.1.0</version> </dependency> <!-- provided: servlet container supplies this -->
<dependency> <groupId>jakarta.servlet</groupId> <artifactId>jakarta.servlet-api</artifactId>
<version>6.0.0</version> <scope>provided</scope> </dependency> <!-- runtime: JDBC driver -->
<dependency> <groupId>org.postgresql</groupId> <artifactId>postgresql</artifactId>
<version>42.7.1</version> <scope>runtime</scope> </dependency> <!-- test --> <dependency>
<groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter</artifactId>
<version>5.10.1</version> <scope>test</scope> </dependency> </dependencies>
```

Transitive Dependencies & Conflict Resolution



Maven resolves transitive dependencies automatically. When conflicts arise (same artifact, different versions), Maven applies the **nearest-wins** strategy: the version closest to the root wins. At equal depth, the **first-declared wins**. Override any transitive version by declaring it explicitly as a direct dependency or in .

Excluding Transitive Dependencies

```
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-logging</artifactId> <!-- Exclude logback, use log4j2 instead
--> <exclusions> <exclusion> <groupId>ch.qos.logback</groupId>
<artifactId>logback-classic</artifactId> </exclusion> </exclusions> </dependency> <dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-log4j2</artifactId> </dependency>
```

dependencyManagement — Centralised Version Control

The **dependencyManagement** section sets default versions and scopes for dependencies. Child modules still declare the dependency but omit the version, inheriting it from the parent. This is the primary mechanism for consistent versioning across multi-module projects.

```
<!-- Parent POM --> <dependencyManagement> <dependencies> <dependency>
<groupId>org.slf4j</groupId> <artifactId>slf4j-api</artifactId> <version>2.0.9</version>
</dependency> <dependency> <groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId> <version>2.16.1</version> </dependency>
</dependencies> </dependencyManagement> <!-- Child module — no version needed -->
<dependencies> <dependency> <groupId>org.slf4j</groupId> <artifactId>slf4j-api</artifactId>
<!-- version inherited --> </dependency> </dependencies>
```

BOM (Bill of Materials) Import

Projects import BOM → declare deps WITHOUT version — BOM manages versions centrally

```
<!-- Import Spring Boot BOM to manage all Spring versions --> <dependencyManagement>
<dependencies> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-dependencies</artifactId> <version>3.2.0</version> <type>pom</type>
<scope>import</scope> </dependency> <!-- Also import Testcontainers BOM --> <dependency>
<groupId>org.testcontainers</groupId> <artifactId>testcontainers-bom</artifactId>
<version>1.19.3</version> <type>pom</type> <scope>import</scope> </dependency> </dependencies>
</dependencyManagement> <!-- Now declare without version --> <dependencies> <dependency>
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-web</artifactId>
</dependency> <dependency> <groupId>org.testcontainers</groupId>
<artifactId>postgresql</artifactId> <scope>test</scope> </dependency> </dependencies>
```

Dependency Analysis Commands

Command	Purpose
mvn dependency:tree	Show full transitive dependency tree
mvn dependency:tree -Dincludes=slf4j	Filter tree to show only slf4j entries
mvn dependency:analyze	Find unused declared & undeclared used deps
mvn dependency:analyze-only	Analyze without building

mvn dependency:list	Flat list of all resolved dependencies
mvn dependency:resolve	Download all deps and print resolved versions
mvn dependency:purge-local-repository	Delete deps from local repo and re-download
mvn versions:display-dependency-updates	Show available version upgrades
mvn versions:use-latest-releases	Update pom.xml to latest release versions
mvn versions:set -DnewVersion=2.0.0	Set project version and all sub-modules

Optional vs Required Dependencies

```
<!-- Optional: consumers choose whether to include this transitive dep --> <dependency>
<groupId>com.google.guava</groupId> <artifactId>guava</artifactId>
<version>32.1.3-jre</version> <optional>true</optional> </dependency> <!--
<optional>true</optional> means: - This library uses guava internally - Consumers that DON'T
need guava features won't get it transitively - Consumers that DO use guava features must
declare it explicitly -->
```

50+ Interview Questions & Answers

Q1. What are Maven's dependency scopes?

compile (default), provided, runtime, test, system, import. Each controls which classpaths (main compile, test compile, runtime) the dependency appears on and whether it's included in the packaged artifact.

Q2. What is the difference between compile and provided scope?

compile: on all classpaths + packaged. provided: on main/test compile classpaths only, NOT packaged — the runtime container (e.g., Tomcat) provides it. Example: jakarta.servlet-api.

Q3. What is runtime scope?

Available at runtime and test runtime, but NOT on the main compile classpath. Typical use: JDBC drivers, which are loaded reflectively. Your code compiles against the JDBC API (compile scope), not the driver.

Q4. What is the test scope?

Available only during test compilation and execution. Not included in the final artifact. Examples: JUnit, Mockito, AssertJ. Does NOT propagate transitively.

Q5. What is the system scope?

Like provided but you specify the JAR path explicitly via . Highly discouraged — not portable, breaks reproducible builds. Use an internal repository instead.

Q6. What is the import scope?

Only valid in . Used with pom to import a BOM, merging its managed versions into the current project's dependency management.

Q7. What is a BOM (Bill of Materials)?

A POM with pom containing only . Consumers import it to inherit a curated set of compatible dependency versions. Spring Boot BOM is the canonical example.

Q8. How do you import multiple BOMs in Maven?

List multiple entries with scope=import in your section. BOMs are merged in declaration order — first declaration wins for conflicts.

Q9. What is the nearest-wins conflict resolution rule?

When two paths to the same artifact exist in the dependency tree, Maven selects the one with fewest hops from the project root. Direct declarations (depth=1) always win.

Q10. What is the first-declared wins rule?

When two conflicting dependencies are at equal depth, Maven selects the one declared first in pom.xml (top-to-bottom order).

Q11. How do you force a specific transitive dependency version?

Declare it as a direct in your pom.xml (overrides via nearest-wins) OR add it to to set the managed version without adding to classpath.

Q12. What is the difference between and ?

adds the artifact to the classpath. only registers version/scope defaults — no classpath effect until the dependency is also declared in .

Q13. How does parent POM work for child modules?

Child modules inherit all entries from parent. They can then declare dependencies without versions — the managed version is applied automatically.

Q14. What does the flag do?

Marks a dependency as optional — it's needed by this library but consumers don't get it transitively. Consumers using features that need it must declare it explicitly.

Q15. What does do?

Removes a specific transitive dependency from the dependency graph for a given direct dependency. Useful for replacing logging implementations or removing conflicting libraries.

Q16. Can you exclude all transitive dependencies at once?

Yes: **. Supported since Maven 3.2.1. Use carefully — you may need to re-declare required transitive deps explicitly.

Q17. What does mvn dependency:tree -Dverbose=true show?

Shows ALL dependency paths including omitted ones (with 'omitted for conflict' or 'omitted for duplicate' labels), useful for debugging version conflicts.

Q18. How do you check for unused dependencies?

mvn dependency:analyze. It uses bytecode analysis to find declared-but-unused and used-but-undeclared dependencies. Some false positives exist for annotation processors and reflection.

Q19. What is the difference between dependency:analyze and dependency:analyze-only?

analyze runs the compile phase first then analyses. analyze-only assumes compilation is already done and only performs the analysis phase.

Q20. How do you see what version of a dependency Maven selected?

mvn dependency:tree -Dincludes=groupId:artifactId. Shows the selected version and the path that won the mediation.

Q21. What is dependency convergence?

All paths to the same artifact resolve to the same version. The maven-enforcer-plugin's DependencyConvergence rule fails the build if any artifact has multiple versions in the tree.

Q22. What is the maven-enforcer-plugin's requireUpperBoundDeps rule?

Ensures that the selected (nearest) version of each dependency is at least as high as what any dependency in the tree declared it needs — prevents using a version too old for some transitive user.

Q23. What is dependency locking (pinning)?

Recording exact resolved versions to a lock file for reproducibility. Maven doesn't have native locking, but the flatten-maven-plugin and the versions plugin help. Gradle has native dependency locking.

Q24. How do you update all dependencies to their latest versions?

mvn versions:use-latest-releases (stable) or mvn versions:use-latest-versions (including alphas). Creates backup .versionsBackup files. Review changes before committing.

Q25. What is the versions:set goal used for?

Sets the version of the project and all sub-modules atomically: mvn versions:set -DnewVersion=2.0.0. Use versions:commit to remove .versionsBackup files afterward.

Q26. How do you resolve a ConvergenceException from the enforcer plugin?

Identify which paths cause the conflict (mvn dependency:tree -Dverbose), then either add an explicit for the desired version, or use to pin it.

Q27. What is the difference between a RELEASE and a SNAPSHOT version?

RELEASE is immutable — once deployed, the artifact should never change. SNAPSHOT is mutable — Maven re-downloads it on every build (within the configured update interval, default: daily).

Q28. How do you force Maven to re-download SNAPSHOT artifacts?

mvn clean install -U (--update-snapshots). Forces download even if the local copy is within the update interval.

Q29. What is the updatePolicy for repositories?

Configures how often Maven checks for updates: always, daily (default), interval:N (minutes), never. Configured in .

Q30. What is a virtual dependency / dependency alias?

Using properties to define versions centrally: 2.16.1 in , then ${jackson.version} in dependencies. Not truly a Maven feature but a strong convention.

Q31. How do you publish an artifact to Nexus/Artifactory?

Configure distributionManagement in pom.xml with repository/snapshotRepository URLs, add credentials in settings.xml , then run mvn deploy.

Q32. What does the localRepository path in settings.xml control?

The path to the local Maven repository. Defaults to ~/.m2/repository. Can be changed per developer or CI to isolate builds.

Q33. What are offline builds in Maven?

mvn -o uses only locally cached artifacts. Fails for any uncached dependency. Useful for air-gapped CI environments where artifacts are pre-seeded.

Q34. How does Maven handle classifier in dependencies?

differentiates artifacts with the same GAV — e.g., tests jar (tests), native binaries (linux-x86_64), javadoc, sources.

Q35. What is the type element in a dependency?

Specifies artifact type: jar (default), pom, war, test-jar, zip, etc. When importing a BOM you need pomimport.

Q36. How do you add a test-jar from another module?

com.exampleshared-test1.0test-jartest. The providing module needs maven-jar-plugin configured with test-jar.

Q37. What is the maven-dependency-plugin:copy-dependencies goal?

Copies all resolved dependencies to a target directory. Useful for creating distribution zips or Docker image layers: mvn dependency:copy-dependencies -DoutputDirectory=target/lib.

Q38. What does dependency:unpack do?

Extracts a specific dependency archive to a directory. Useful for unpacking native libraries or config files packaged as JARs.

Q39. What is a fat JAR (uber-jar)?

A single JAR containing all classes from the project and all its dependencies. Created by maven-shade-plugin or spring-boot-maven-plugin (repackage goal). Suitable for standalone deployment.

Q40. What is package relocation in maven-shade-plugin?

Rewrites package names of bundled dependencies to avoid classpath conflicts: relocate guava from com.google.common to shaded.com.google.common. Prevents split-package issues when multiple versions coexist.

Q41. What is the maven-assembly-plugin?

Creates distribution archives (zip, tar.gz) bundling the project JAR plus dependencies, config files, scripts. More flexible than shade but produces a directory structure rather than an uber-jar.

Q42. How do you prevent accidental inclusion of SNAPSHOT dependencies in releases?

Use maven-enforcer-plugin with the requireReleaseDeps rule: . Fails the build if any SNAPSHOT dependency is found.

Q43. What is the Maven local repository cache structure?

~/.m2/repository/{groupId path}/{artifactId}/{version}/{artifactId}-{version}.jar + .pom + .sha1. Maven stores all resolved artifacts here for offline reuse.

Q44. How do you add a local file system JAR as a Maven dependency?

Option 1 (preferred): mvn install:install-file to install it to local repo. Option 2: use system scope with ${project.basedir}/lib/mylib.jar. System scope is not recommended for shared builds.

Q45. What is the difference between mvn dependency:resolve and dependency:tree?

resolve downloads all artifacts and prints a flat list of what was resolved. tree shows the full hierarchical dependency graph including transitive dependencies and how they were pulled in.

Q46. What does the dependencyManagement import override precedence look like?

Direct dependencyManagement entries take precedence over imported BOM entries. Between multiple BOMs, declaration order matters — first import wins for version conflicts.

Q47. How do you downgrade a transitive dependency version?

Declare it explicitly in with the older version (nearest-wins overrides transitive version) OR pin it in . Maven will use your declared version.

Q48. What is the maven-versions-plugin enforceSnapshots goal?

Checks that SNAPSHOT dependencies have been replaced with release versions — typically run as part of a release gate to ensure production builds don't reference mutable artifacts.

Q49. How do you handle a split-package problem in Java 9+ module system with Maven?

Identify overlapping packages using mvn dependency:tree + module-info analysis. Use maven-shade-plugin with relocation to move conflicting packages, or exclude the duplicate and use the correct artifact.

Q50. What happens if you declare the same dependency twice in pom.xml?

Maven uses the last declaration (for the same GAV). This is a warning-worthy practice — keep only one declaration and use dependencyManagement for version control.

Q51. What is the maven-enforcer-plugin bannedDependencies rule?

Prevents specific artifacts from being on the classpath: commons-logging:commons-logging. Useful for enforcing a preferred logging bridge.